Case study · November 2026
Prepared by Anoop Singh
github.com/devbyanoop/graphtrace

Building graphtrace: what it takes to make agent testing actually trustworthy

A short account of five real bugs found while building this tool — and what each one says about the discipline applied to it, written for anyone deciding whether to bring that same discipline to their own project.

OPEN SOURCE · APACHE 2.0 | LIVE ON PyPI | 222 TESTS PASSING | VERIFIED ON 2 REAL EXTERNAL APPS

The problem it solves

A LangGraph agent with branches, retries, and tool calls can look fine on a lucky run while silently changing *how* it got there — a node path that changed, a retry now firing on every call, a tool call that quietly stopped happening, an approval gate the agent learned to route around. Checking only the final answer text can't catch any of that, because the final answer can still look right while the path that produced it is wrong, expensive, or unsafe.

graphtrace records the full trajectory of a run — every node, every state change, every tool/LLM call — and makes that path a first-class, assertable thing, with record/replay cassettes so a CI run never has to make a real, billed LLM call to pass.

222	12	25	5
TESTS PASSING	ASSERTION FUNCTIONS	REAL-APP SCENARIOS	REAL BUGS FOUND & FIXED

What this case study is actually about

Not the feature list — that lives in the README. This is about five real bugs that building graphtrace surfaced, each one found by deliberately checking a claim against real behavior instead of trusting a design document's description of it — and what that says about how this kind of work gets done.

> **1. The attribution mechanism in the original design didn't survive real concurrency**
>
> The plan called for a thread-local variable set to "the current node" while a graph streamed, read back inside whatever LLM/tool call that node made. It looked right. It wasn't: LangGraph runs parallel nodes on worker threads that don't inherit the calling thread's context by default. Confirmed with a live thread probe inside a real fan-out node before trusting the fix — not after. The real answer was better anyway: LangGraph already tags every call with its own node/task id in metadata that travels with the call through whatever thread actually executes it, no side channel needed.

> **2. The chat-model patch target didn't intercept anything**
>
> The original design patched a base class method. Every real chat model — OpenAI's, Anthropic's, LangChain's own test doubles — defines its own version that overrides the base class's. Patching the base

class patches a method nothing ever calls. Found by checking the real class dictionary directly, rather than assuming the architecture document's description was current.

3. A YAML feature broke a downstream library in a way neither library's own docs would have predicted

The YAML library's round-trip mode — needed so a growing recording file diffs cleanly in a pull request — wraps a loaded number in a subclass that preserves its exact on-disk formatting. That same value, replayed back into a real LangGraph run, flows into the framework's own state and gets serialized by its checkpointer, which doesn't recognize the subclass and raises. Two correct libraries, one real incompatibility, found only by running the actual round trip end to end.

4. A statistics claim in the project's own build plan was simply wrong

The plan asserted that scoring a human-vs-AI-judge agreement set with Cohen's kappa gives 0.0 for "full disagreement." For balanced labels, a *systematic* inversion actually scores -1.0 — a real, measurable anti-correlation, worse than chance. Confirmed by deriving the formula by hand and checking it against the real, installed statistics library, not by re-reading the plan more carefully. The distinction matters: a score near -1 says "the judge may be reading this rubric backwards," which is a sharper, more actionable signal than a score near 0, "the judge adds no information."

5. A feature promised since day one was never actually built — and nothing caught it for weeks

The data model declared two step types for entering and exiting a sub-agent from the very first week. Nothing in the recorder ever produced them, because nothing had ever tested a node whose value is itself a compiled sub-graph — a genuinely common pattern in real multi-agent systems. Investigating it by hand surfaced a second, more serious problem underneath: the recorder's internal state tracking was global, not scoped per sub-graph, so one sub-agent running concurrently with another node could silently corrupt that sibling's recorded data. Reproduced deliberately, fixed by scoping the bookkeeping correctly, and confirmed with tests that were each checked to genuinely fail against the pre-fix code — not just pass against the fix.

Why this is the point, not a footnote. Every one of these was in code that already had passing tests. None would have been caught by writing more tests against the same assumptions, or by reading the design documentation more carefully — the documentation was wrong, or silent, in exactly the way that looks fine until someone deliberately runs the real thing and checks. That is the discipline behind this project: pin down what a library, a statistic, or a framework's own promise genuinely does, by running it, before building anything on top of the belief that it does what it's supposed to.

Where this leaves graphtrace

graphtrace is a complete v0.1, live on PyPI: the trajectory model and its 12-function assertion API, real LangGraph recording (including sub-graphs), record/replay cassettes, an LLM-as-judge contract with calibration against a human-labelled set, a CLI that turns a production trace export into a generated test, a static HTML report with a go/no-go executive summary, and CI — all verified against real upstream sources, not just mocks, including two real, official external LangGraph applications across 25 scenarios. v0.2 (a

security/red-team test pack, observability export, cut-point replay) and v0.4 (CrewAI and AutoGen support) are scoped and not yet started.

If this is the kind of rigor you want applied to testing, auditing, or hardening your own LangGraph or agentic AI system —

Code & full build history: **github.com/devbyanoop/graphtrace** · Package: **pypi.org/project/graphtrace** · Contact: **github.com/devbyanoop**

graphtrace is open source under the Apache License 2.0. This case study describes graphtrace v0.1.0.